Trees

Deep Dive + Real-World Use Cases + Complexity

Java 8+

Session Structure

1. Tree fundamentals & terminology
2. Binary Tree structure
3. Traversals (DFS & BFS)
4. Time & space complexity
5. Recursive vs Iterative approaches
6. Real-world applications
7. Interview-level problems

What is a Tree?

A Tree is a:

- Hierarchical data structure
- Non-linear
- Composed of nodes

Unlike arrays or lists:

- No sequential order
- Parent-child relationship

Tree Terminology

- Root
- Parent
- Child
- Leaf
- Height
- Depth
- Subtree

Height of tree = longest path from root to leaf.

Binary Tree

Each node has at most:

- Left child
- Right child

Structure:

```
    A
   / \
  B   C
 / \
D   E
```

Tree Node Structure (Java)

```java
class TreeNode {
    int val;
    TreeNode left, right;

    TreeNode(int val) {
        this.val = val;
    }
}
```

Why Trees?

Efficient for:

- Hierarchical data
- Expression parsing
- File systems
- Organizational charts
- JSON / XML parsing

Depth First Traversals (DFS)

1. Preorder → Root, Left, Right
2. Inorder → Left, Root, Right
3. Postorder → Left, Right, Root

Time Complexity: O(n)

Space Complexity: O(h) (recursion stack)

Preorder Traversal

```
static void preorder(TreeNode root) {
    if (root == null) return;

    System.out.print(root.val + " ");
    preorder(root.left);
    preorder(root.right);
}
```

Inorder Traversal

```
static void inorder(TreeNode root) {
    if (root == null) return;

    inorder(root.left);
    System.out.print(root.val + " ");
    inorder(root.right);
}
```


Postorder Traversal

```
static void postorder(TreeNode root) {
    if (root == null) return;

    postorder(root.left);
    postorder(root.right);
    System.out.print(root.val + " ");
}
```


Breadth First Traversal (Level Order)

Uses Queue.

Time: O(n)

Space: O(n)

```
static void levelOrder(TreeNode root) {
    if (root == null) return;

    java.util.Queue<TreeNode> queue = new java.util.ArrayDeque<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        TreeNode node = queue.poll();
        System.out.print(node.val + " ");

        if (node.left != null) queue.offer(node.left);
        if (node.right != null) queue.offer(node.right);
    }
}
```


Height of Binary Tree

Time: O(n)

Space: O(h)

```
static int height(TreeNode root) {
    if (root == null) return 0;

    return 1 + Math.max(height(root.left), height(root.right));
}
```


Real World Use Case — File System

```
Root
 ├── Documents
 ├── Downloads
 └── Pictures
```

Hierarchy stored as tree.

Real World Use Case — Expression Tree

Expression:

```
(3 + 5) * 2
```

Tree:

```
      *
     / \
    +   2
   / \
  3   5
```


Maximum Depth Problem

```java
static int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}
```

Time: O(n)

Check if Two Trees Are Same

```
static boolean isSame(TreeNode p, TreeNode q) {
    if (p == null && q == null) return true;
    if (p == null || q == null) return false;

    return p.val == q.val &&
           isSame(p.left, q.left) &&
           isSame(p.right, q.right);
}
```

Time: O(n)

Path Sum Problem

Time: O(n)

```
static boolean hasPathSum(TreeNode root, int sum) {
    if (root == null) return false;

    if (root.left == null && root.right == null)
        return sum == root.val;

    return hasPathSum(root.left, sum - root.val) ||
           hasPathSum(root.right, sum - root.val);
}
```


Iterative DFS Using Stack

```
static void preorderIterative(TreeNode root) {
    if (root == null) return;

    java.util.Stack<TreeNode> stack = new java.util.Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        System.out.print(node.val + " ");

        if (node.right != null) stack.push(node.right);
        if (node.left != null) stack.push(node.left);
    }
}
```


Tree Complexity Summary

Operation	Time
Traversal	O(n)
Height	O(n)
Search	O(n)
Space (recursion)	O(h)

Where:

- n = number of nodes
- h = height of tree

Common Mistakes

- Forgetting null checks
- Confusing height vs depth
- Stack overflow in skewed tree
- Not handling leaf correctly

Interview Discussion Points

- Difference between DFS and BFS?
- When does recursion cause stack overflow?
- Balanced vs skewed tree?
- Why traversal is always O(n)?

Practice Problems

Easy:

- Maximum depth
- Invert binary tree

Medium:

- Diameter of tree
- Balanced tree check

Advanced:

- Serialize & deserialize tree
- Lowest common ancestor
- Vertical order traversal

Summary

Trees provide:

- Hierarchical modeling
- Efficient recursion patterns
- Foundation for BST & Heap
- Core of advanced algorithms

Master:

- DFS (Pre, In, Post)
- BFS (Level order)
- Height calculation
- Path problems